Project's Report

on

Marketing Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Aditi Gaikwad	EN22CS301040
2	Akhilesh Tiwari	EN22CS301085
3	Alfez Khan	EN22CS301102
4	Akshat Sakalye	EN22CS301093
5	Aanchal Chaturvedi	EN22CS301014
6	Aditya Patidar	EN22CS301060

Group Name: Group 06D1

Project Number: AAI-06

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

1 Problem Statement & Objectives

1.1 Problem Statement

Modern marketing teams struggle to manually analyze competitor data, validate budgets, check resource availability, and format actionable schedules in a timely manner. Traditional single-prompt Large Language Models (LLMs) fail at automating this process because they lack step-by-step reasoning loops and often hallucinate constraints. There is a critical need for an autonomous system that can reliably decompose high-level marketing goals, consult accurate data sources, and generate structured execution plans without human micro-management.

1.2 Project Objectives

1. **Develop an Autonomous Orchestrator:** Build a multi-agent backend orchestrator using a Plan-and-Execute reasoning loop to decompose high-level marketing goals into actionable tasks.
2. **Implement Role-Specific AI Agents:** Utilize a PlannerAgent to determine the execution sequence and delegate tasks, alongside a SchedulerAgent to synthesize aggregated data into a formatted Markdown document.
3. **Ensure Data Isolation & Security:** Create a strict multi-tenant architecture using JWT authentication and relational databases so that different organizations' data and generated plans remain completely isolated.
4. **Integrate Real-World Tooling:** Equip the agents with dynamic tools (such as vector database search for RAG and mock APIs for budget/resource validation) to ensure generated strategies are grounded in real data rather than LLM hallucinations.

1.3 Scope of the Project

The scope of this project encompasses the end-to-end development of an autonomous, multi-tenant Software-as-a-Service (SaaS) application. It involves building a secure backend infrastructure capable of handling JWT-based authentication and organization-level data isolation. The core AI scope includes designing a multi-agent orchestration layer that utilizes a Plan-and-Execute reasoning loop, allowing specialized agents to

interact with local vector databases (ChromaDB) and external tools. The frontend scope is limited to a React-based Single Page Application (SPA) dashboard that provides users with interface controls to upload context documents, submit marketing goals, and manage their generated execution schedules.

2 Proposed Solution

We propose a Modular Architecture inspired by Clean Architecture principles, completely decoupling the frontend presentation layer from the AI orchestration engine. Instead of relying on a standard deterministic MVC pattern or a single API call to a Large Language Model, our solution introduces a "Service Layer" managed by CrewAI. This orchestrator assigns specific tasks to specialized AI personas (Strategist, Execution Planner, and Content Publisher), routing complex reasoning tasks to heavy-duty models (Groq Llama 3.3 70B) and formatting tasks to faster models (OpenRouter Llama 3 8B). This ensures high-quality, hallucination-free outputs grounded in user-provided context.

2.1 Key Features

- **Strict Multi-Tenancy:** Complete data isolation by Organization using JWTs and relational database constraints.
- **Multi-Agent Orchestration:** Specialized AI personas collaborating sequentially to analyze, plan, and format data.
- **Retrieval-Augmented Generation (RAG):** Local ChromaDB vector storage for analyzing uploaded competitor data and strategy PDFs.
- **Persistent Plan History:** Automated MySQL database saving for reading, renaming, and deleting past generated schedules.
- **Resilient API & UI:** Global error interceptors, local-storage-backed Dark/Light mode, and safe semantic HTML rendering.

2.2 Overall Architecture/Workflow

The system workflow follows a sequential, decoupled client-server loop:

1. **Input:** A user securely logs into the React frontend and submits a high-level marketing goal alongside optional PDF context documents.

2. **API Transport:** The FastAPI backend receives the payload, validates the user's JSON Web Token (JWT), and extracts the tenant's organization_id.
3. **Orchestration & RAG:** The backend initiates the CrewAI process. The *Strategist Agent* searches the ChromaDB vector store (strictly filtered by org_id) to gather context and formulates a strategic approach.
4. **Planning & Execution:** The strategy is passed to the *Execution Planner Agent*, which decomposes the goal into a structured Markdown schedule.
5. **Formatting & Output:** The *Content Publisher Agent* uses a custom Python tool to convert the Markdown into semantic HTML.
6. **Persistence:** The FastAPI controller intercepts the final HTML, atomically saves it to the MySQL database linked to the user's organization, and returns the payload to the frontend for display.

2.3 Tools & Technologies Used

The platform was built using a decoupled client-server architecture, leveraging modern web frameworks and advanced AI orchestration tools:

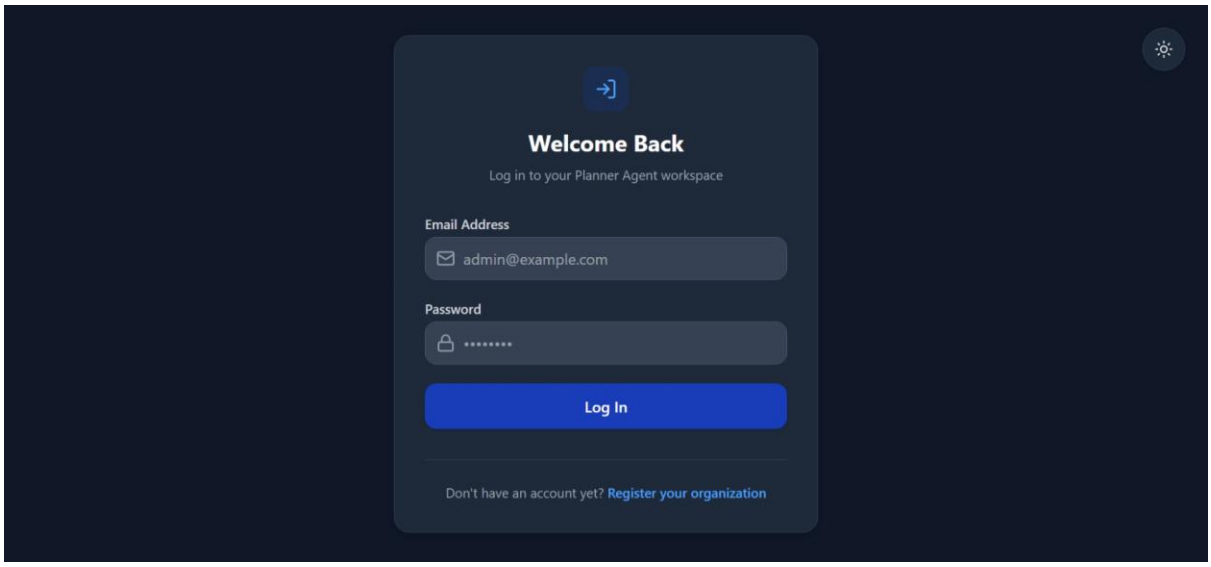
- **Frontend (Client):** React 18, Vite, Tailwind CSS v4, Axios (for API requests), and Lucide React (for iconography).
- **Backend (API Server):** Python 3.10+, FastAPI, Uvicorn (ASGI server), SQLAlchemy (ORM), PyMySQL, PyJWT, and bcrypt (for security).
- **AI & Orchestration:** CrewAI (for multi-agent framework), LiteLLM (for provider routing), Groq API (llama-3.3-70b-versatile), and OpenRouter API (meta-llama/llama-3-8b-instruct).
- **Database & RAG:** MySQL (for relational multi-tenant data), ChromaDB (for local vector storage), and HuggingFace sentence-transformers (all-MiniLM-L6-v2 for text embeddings).

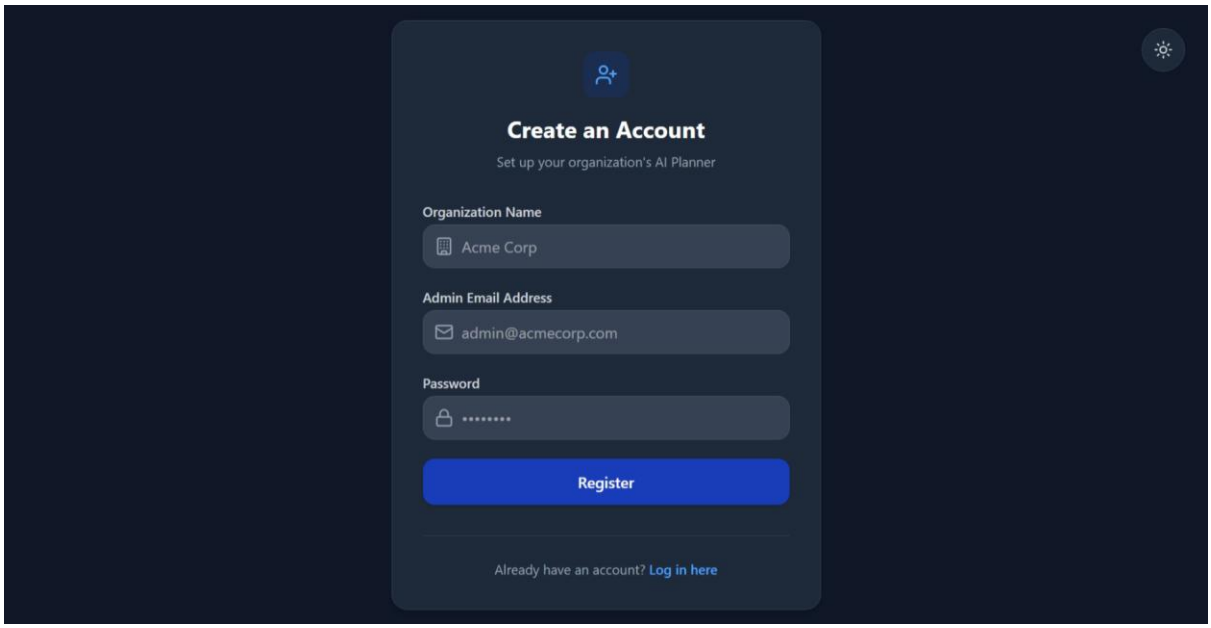
3 Results & Output

3.1 Screenshots / outputs

3.1.1 Figure 1: User Authentication & Tenant Creation

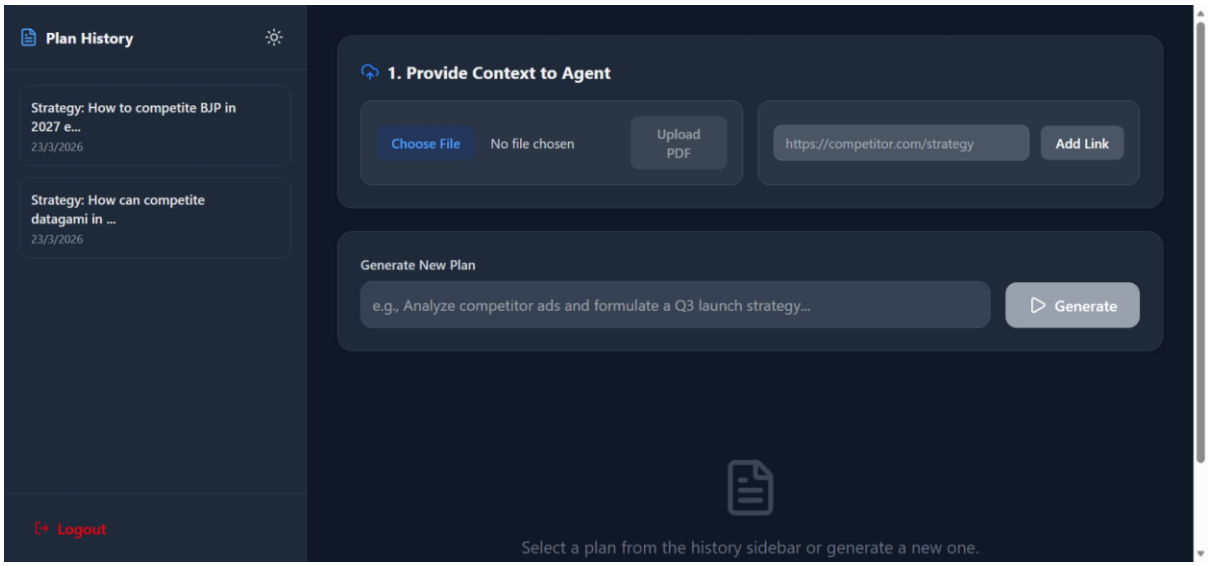
- *Description:* The secure Login and Registration pages demonstrating JWT-based access control.





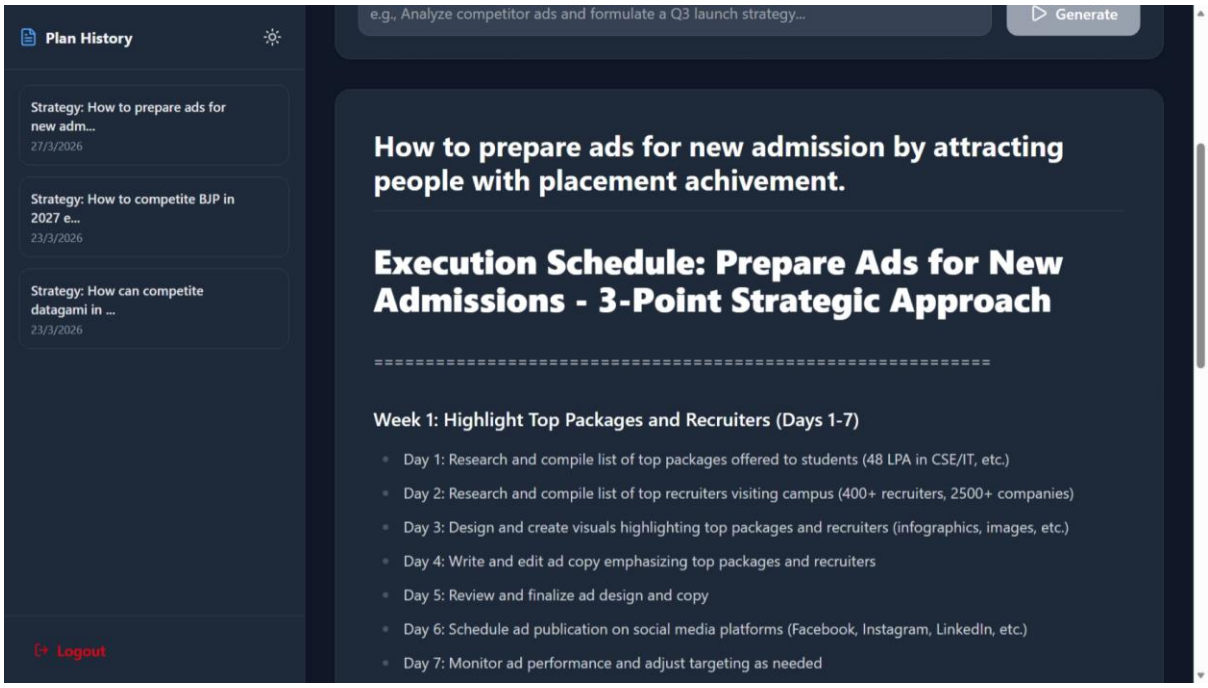
3.1.2 Figure 2: The SaaS Dashboard & Knowledge Base Ingestion

- *Description:* The main UI where users can upload context PDFs (processed by ChromaDB) and submit their marketing goals.



3.1.3 Figure 3: Agentic Execution & Formatted Output

- *Description:* The final semantic HTML execution schedule successfully rendered by the frontend, alongside the History Sidebar showing previously auto-saved plans.



3.2 Reports / dashboards / models

The core model of this project is the **Agentic Orchestration Engine**. Unlike standard chatbots, our model successfully demonstrates a multi-agent workflow where a

Strategist Agent retrieve vectorized constraints, an *Execution Planner Agent* schedules the tasks, and a *Content Publisher Agent* handles strict HTML formatting. The user-facing dashboard effectively hides this complex backend logic, providing a seamless SaaS experience with persistent state management (Dark/Light mode) and historical data retrieval.

3.3 Key outcomes

1. **Successful Data Isolation:** The system successfully implements strict multi-tenancy. Database queries and Vector DB RAG searches correctly filter by organization_id, ensuring no data leakage between different user accounts.
2. **Elimination of Formatting Hallucinations:** By separating the planning logic from the formatting logic and assigning the HTML conversion to a heavy-duty model (70B) paired with a deterministic Python tool, the system consistently outputs structurally perfect documents.
3. **Autonomous Workflow Validation:** The CrewAI implementation proved that autonomous agents can successfully hand off state and context sequentially without requiring human intervention midway through the generation process.

4 Conclusion

The Marketing Planning Assistant Agent successfully bridges the gap between advanced Large Language Model orchestration and practical, enterprise-grade web development. By decomposing complex marketing strategies into a multi-agent Plan-and-Execute loop, the system provides highly actionable, formatted, and context-aware schedules.

The key learning from this project was mastering the integration of non-deterministic AI outputs within a strictly typed, deterministic software architecture. Our team successfully navigated the challenges of API rate limiting, LLM context window collapse, and secure multi-tenant database management, ultimately delivering a robust, end-to-end SaaS platform.

5 Future Scope & Enhancements

While the current v1.0 architecture is fully functional, the system can be expanded with the following enhancements:

1. **Live Web Scraping Tools:** Develop a custom Python tool using BeautifulSoup to allow the Strategist Agent to ingest live competitor URLs directly from the internet, rather than relying solely on uploaded PDFs.
2. **Semantic Caching:** Implement a caching layer (e.g., Redis) at the API gateway to store previous LLM responses for highly similar marketing goals, drastically reducing API costs and latency for redundant queries.
3. **Agent Roster Expansion:** Add highly specialized agents to the Crew, such as an "SEO Specialist Agent" to optimize the generated copy, or a "Financial Agent" to strictly calculate and allocate simulated budget constraints.
4. **Cloud Vector Migration:** Transition the local ChromaDB instance to a managed, serverless vector database (like Pinecone or Weaviate) to support horizontal scaling in a production cloud environment.